autofill-audit: the debug report

What went wrong, how it was found, and what was done about it

Grouped by theme rather than by date

Olajide Badejo

2026-08-27

Abstract

This is the record of the defects, surprises and reversed decisions of the project, drawn from an engineering log that was written as the work happened rather than reconstructed afterwards. It is grouped by theme, because chronology is how the problems were met and theme is how they are useful to anyone else.

Every entry states the symptom, how it was diagnosed, what was done, and what else was considered and refused. The fourth of those is the part that is ordinarily missing from a defect record and is usually the reason the entry was worth keeping: a fix without its rejected alternatives is a fact, and a fix with them is an argument somebody can disagree with.

The document is a first class deliverable rather than an appendix. The honest record of what went wrong is the part of a project that ordinarily evaporates, and almost nobody publishes one.

Contents

Chapter 1

DOM traversal and shadow boundaries

The traversal itself was right almost immediately. Everything in this chapter is a case where the DOM said something true and the code drew the wrong conclusion from it.

A label attached itself to the wrong control

Symptom. On a fixture with two elements carrying the same identifier, a control with no label of its own was reported with somebody else's label text, and the classification that followed was confident and wrong.

Diagnosis. The first implementation built a map from identifier to label and handed the same label to every element carrying a duplicated identifier. That is not what the platform does. A label's `for` attribute names exactly one element, the one the document's own lookup would return, and a duplicated identifier does not make two elements labelled. It makes one labelled and one not.

Fix. Resolve the association the way the platform resolves it: first element wins, everything else has no label from that source and falls through to the next source in the precedence order.

Considered and refused. Reporting a duplicate identifier as a defect and refusing to associate either. That is a real finding and it is emitted, but it is a warning about the page rather than a reason to withhold a label from the element that legitimately owns it.

The canvas rule would have unbalanced every hostile checkout

Symptom. The specification says a canvas rendered form like widget is reported as undetectable rather than guessed at. Implemented literally, that emitted one synthetic descriptor per large canvas, and every hostile checkout form in the corpus then reported one more field than its answer key held. The extraction reconciliation the phase gate asks for could never balance.

Diagnosis. The specification describes a page whose form *is* a canvas. The corpus contains pages with plenty of real controls that also happen to carry a canvas, which is a different shape, and the rule was being applied to both.

Fix. A canvas is always recorded as a region, on every page. The page level undetectable descriptor is emitted only when the page yielded no controls at all. The information is present either way; what changes is whether it counts as a field.

Considered and refused. Excluding synthetic descriptors from the reconciliation. That would have made the gate pass by making the gate weaker, and the gate exists to catch exactly the

class of error where the extractor and the answer key disagree about how many fields a page has.

The frame accessibility test was asking the wrong question

Symptom. The plan assumed the browser automation layer would fail to evaluate inside a cross origin frame, and that this failure was how the tool would detect one.

Diagnosis. It does not fail. The automation layer drives each frame through its own session, so it can evaluate inside a cross origin frame perfectly well. The thing that makes such a frame unreadable to *this tool* is not the automation layer. It is the page's own same origin policy, and a tool that reads across it would be reporting on a document the page cannot see either.

Fix. Make the accessibility test from inside the parent document, by asking for the frame's document in a guarded block. That is exactly the check a script on that page would make, and it is what the specification means by *cannot be accessed*.

Considered and refused. Using the automation layer's cross frame access and reporting on the hosted document. It would have produced more findings and every one of them would have been about a document the page's own author cannot reach from their markup, which is advice nobody can act on.

Two offline fixtures for a problem that seemed to need a server

Symptom. Testing frame behaviour appeared to require two origins, which appeared to require a network or at least two local servers, which the testing rules forbid.

Diagnosis. Origins are cheaper than they look. A frame declared with inline document markup inherits its parent's origin and so is same origin even under a file scheme, where two separate local files are opaque origins to each other and would not be. A frame loaded from a data address gets an opaque origin and so is unreadable.

Fix. Two fixtures, both offline, both reproducing their case exactly, with the reasoning recorded in the environment document so nobody replaces them with a server later.

Considered and refused. A local server fixture in the test suite. It would have worked and it would have made the whole suite depend on a port being free, which is the kind of dependency that turns a green suite into an intermittently red one on somebody else's machine.

One selector, four times, for a group of four radios

Symptom. A radio group produced the same selector once per member.

Diagnosis. The selector preference order includes a rank that selects on the control's name attribute. Every member of a radio group shares one name, so that rank produces one selector for the whole group and hands it to each member. The sketch of the preference order in the specification does not mention this, because the case does not arise for the controls it was written against.

Fix. Rank two requires the name to be unique within the form. A group falls through to the next rank, which distinguishes its members.

Considered and refused. Special casing radio groups in the selector generator. The uniqueness requirement is the general statement of the same rule and it covers checkbox groups and any other shared name case without a second branch.

A child combinator that matched nothing

Symptom. The second rank of the selector order, written literally from the specification's sketch as a form element followed by a child combinator, resolved to no element on every form the generator emits.

Diagnosis. A child combinator matches only a control that is an immediate child of the form element. Every form this generator emits, and essentially every form on the web, wraps its controls in layout containers.

Fix. A descendant combinator. The deviation is recorded in the selector module and in the handoff notes, because the extractor has to reproduce the generator's selector convention exactly rather than approximately, and a difference between the two would silently lose fields.

Considered and refused. Emitting forms without layout containers so that the literal reading works. That would make the corpus unrepresentative of the thing being measured in order to preserve a sketch.

Field order disagrees on exactly one tier, and it is not a defect

Symptom. Reconciling extracted descriptors against answer keys showed an order difference on the mixed markup tier and nowhere else.

Diagnosis. The extractor returns document order, which the traversal specification requires. An answer key lists its fields in the generator's slot order, which appends a tier's extra controls to the end of the list. On the mixed tier a hostile block sits in the middle of a form whose extra controls belong to that block, so they render in the middle and the key lists them last.

Fix. The reconciliation compares selectors as a set and reports order differences separately. The selectors are the contract; the order of the list is not. That every affected form is on the mixed tier is asserted rather than assumed, so a future order difference somewhere else fails the check.

Considered and refused. Sorting both sides before comparing. That would have hidden a real future defect: an extractor that returned fields in an order unrelated to the document would pass a sorted comparison forever.

Five signals with nowhere to go

Symptom. The signal collection specification names several signals that the descriptor schema has no field for.

Diagnosis. Two sections of the specification disagree, and both are load bearing: the collection list is what the traversal should gather, and the descriptor is the fixed downstream contract that every golden test inherits.

Fix. Collect them, keep them on the raw record, and stop before the descriptor. One of them, whether a select accepts multiple values, turned out to be genuinely useful: a multi select with twelve options is a month picker to a careless detector and a multiple choice control to a careful one.

Considered and refused. Widening the descriptor. That is a schema change that invalidates every committed fixture expectation, and it should be made when something needs the field rather than when something collects it.

Chapter 2

Waiting, flake, and process isolation

The most expensive twenty minutes of the project were spent looking in the wrong place because an error message named a symptom that had nothing to do with its cause. That entry is first.

An error message about the wrong API

Symptom. Adding a session scoped browser fixture to the audit test suite made the extractor test suite fail with a message saying the synchronous browser API was being used inside an asynchronous event loop, and recommending the asynchronous API instead. Nothing anywhere was using the asynchronous API.

Diagnosis. The synchronous wrapper drives a coroutine scheduler on one event loop per thread. A second live browser session in the same thread finds that loop already running and reports it in the only vocabulary it has, which is the vocabulary of the asynchronous API. The message is accurate about the state it found and useless about how the state arose.

Fix. One browser fixture in the root test configuration, shared by every suite.

Considered and refused. Making each suite's fixture function scoped. That would have paid a browser launch per test, and it would have left the same landmine for the next person who added a second session concurrently rather than sequentially.

The end to end tests could not run the tool

Symptom. Following on from the entry above: the command line tool opens a browser of its own, and it cannot do that while the test session's browser is alive in the same thread.

Diagnosis. Same root cause, different consequence. It is not a defect in the tool; it is a property of the synchronous wrapper.

Fix. The end to end tests run the tool in a subprocess. That turned out to be an improvement rather than a workaround: a subprocess exercises the real exit codes as a shell sees them, which is the actual contract the tool publishes, and an in process call would have tested a function's return value instead.

Considered and refused. Running the end to end tests in a separate thread. It would have worked and it would have tested a code path no user ever takes.

Choosing waiting constants that are neither flaky nor slow

Symptom. Not a defect so much as a set of decisions that would have become defects if they had been guessed. A page that never settles must not hang the tool, and a page that is merely slow must not be cut off mid load.

Diagnosis. Each constant is answering a different question and they should not be derived from one another.

Fix. Each is a named constant with the reasoning beside it:

- A generous load timeout, because a slow page is a real page, and a bound on it, because a page that never loads must not hang a command line tool.
- A network idle wait that gives up rather than failing, because plenty of healthy pages hold a socket open forever and giving up on idle is not an error.
- A DOM quiet period longer than an animation frame and shorter than the gap a page leaves before injecting a field it means a user to see.
- A total settle budget, so that an animation which mutates forever cannot hold the quiet period off indefinitely.
- A cap on controls, well above any real form, reported when reached and never silent.
- A frame depth bound, because three deep is an ordinary advertising stack and eight is pathological.

Considered and refused. One timeout for everything, tuned until the suite went green. That is how a project acquires a constant nobody can explain and nobody dares change.

A window that would expire

Symptom. A card expiry select has to offer years near the present, and the extractor detects an expiry pair against the current year. A hardcoded window would have made the corpus regenerate differently once the year turned, and the reachability check and the committed sample would both have gone red on a calendar boundary with no code change.

Diagnosis. The corpus is required to be a deterministic function of its seed. A clock is not a function of a seed.

Fix. The base year is a generator argument defaulting to the current year and recorded in the manifest. The corpus is a deterministic function of the seed and the base year together, and passing the manifest's base year back reproduces the bytes exactly, indefinitely. The reachability check and the committed sample both pin it.

Considered and refused. Freezing the year to a literal. The corpus would then have been reproducible and progressively less realistic, and the expiry detection it exists to exercise would have stopped being exercised.

The whitespace hook ate the golden snapshots

Symptom. The first commit of the terminal renderer's golden snapshots rewrote every one of them and left the test suite failing against its own repository.

Diagnosis. The terminal rendering library pads a table row out to the full terminal width. The pre-commit trailing whitespace hook trims it. Both are behaving correctly and the combination is a snapshot that can never match.

Fix. The hook skips the golden directory. That is a narrow exemption for the one directory whose entire purpose is to be byte exact.

Considered and refused. Stripping trailing whitespace when writing and when comparing snapshots. That hides a real class of rendering defect, which is a renderer that starts emitting different padding, and the whole point of a golden snapshot is to notice exactly that.

Chapter 3

Text normalisation and the rule vocabulary

The unglamorous layer, and the one where the only genuinely subtle defects of the extractor lived. Two of the entries below were found by a property test in about two seconds each, and neither would have been found by reading the code.

The published step order destroys its own worked example

Symptom. Normalisation is specified as compatibility normalisation, then case folding, then camel case splitting, then delimiter splitting, then a stoplist. Implemented in that order, the specification's own worked example becomes unreachable: the input folds to a single lowercase run in which no case boundary exists, so the boundary step has nothing to find.

Diagnosis. Two steps are in the wrong order relative to each other. Everything the fold is specified for is a per character property, so it does not need to happen before the boundary pass; the boundary pass genuinely does need the case information the fold destroys.

Fix. Compatibility normalisation, then boundary insertion, then fold each resulting token. Nothing the fold exists for is lost. What is gained is that the boundary step works at all.

Considered and refused. Following the order literally and adjusting the worked example. That would have been transcription rather than reading, and the resulting tokeniser would have been useless on the camel case identifiers that are most of what a hostile page leaves standing.

Combining marks were being treated as separators

Symptom. A property test asserting that normalisation is idempotent failed almost immediately, on the Turkish dotted capital letter I.

Diagnosis. The splitter was built on the language's word character class, which excludes combining marks. Folding the Turkish letter yields a base letter followed by a combining dot; splitting on that dot drops the dot; a second pass over the same text therefore returns something different from the first.

Fix. Treat combining marks as part of the token they attach to.

Considered and refused. Narrowing the property to the six locales the corpus covers and calling idempotence a practical guarantee. This is the entry that most nearly went the wrong way, and the reason it did not is worth keeping. Splitting on combining marks shreds Devanagari, Thai, pointed Hebrew and anything in decomposed form. None of that appears in this project's corpus, so the narrowed property would have passed forever, and the defect would have been

waiting for the first person to point the tool at a page outside those six locales. The strategy went the other way instead and now sweeps the whole basic multilingual plane.

Case folding does not always produce lowercase

Symptom. The same property test failed again on the next run, on a different input.

Diagnosis. Cherokee folds upward. Folding a lowercase Latin letter beside a Cherokee capital therefore leaves a lowercase letter next to an uppercase one: a case transition that the camel case pass, which runs before the fold, never saw. Feed the output back in and it splits into two tokens, which is a different answer from the first.

Fix. Bracket the fold with boundary insertion rather than running it once before. Folding before the boundary pass is not an option, because that is the defect in the entry above, so the only order satisfying both constraints is to insert boundaries, fold, and insert boundaries again.

Considered and refused. Excluding scripts that fold upward. There is one obvious such script and assuming it is the only one is exactly the assumption that produced this defect in the first place.

A stoplist entry that could never match

Symptom. The one entry on the stoplist that names a real framework's generated identifier prefix never fired.

Diagnosis. It cannot survive the letter to digit split, which turns it into a letter fragment and a digit fragment. A stoplist applied only after splitting would never see it in the form the list is written in.

Fix. Apply the stoplist twice, to the delimiter separated fragments first and to the finished tokens second, from one list.

Considered and refused. Adding the split halves to the list. The halves are both extremely common substrings and adding them would have removed real signal from unrelated identifiers.

A German placeholder read as a whole name field

Symptom. On the hostile tier the label is stripped and the placeholder is all that survives. The German email placeholder is the conventional documentation address, the at sign does not survive normalisation, so the token stream reads as three ordinary words, and the whole name rule fired on the first of them. Ten confident wrong criticals across the hostile slice.

Diagnosis. The rule table had no way to recognise a placeholder that is an email address once the punctuation had been removed.

Fix. A rule matching the reserved documentation domain of the relevant standards document, which is what essentially every placeholder on the web uses.

Considered and refused. Removing the whole name rule's weakest pattern, or special casing the German locale. The domain rule turned ten wrong criticals into ten right ones across the entire hostile slice rather than fixing the German case, because the same placeholder convention is used in every locale in the corpus.

A Japanese section heading read as a card verification code

Symptom. A login password field on a Japanese form picked up a card security code classification from its surrounding context.

Diagnosis. The rule for the Japanese security code matched the bare word for *security*, which is what a form's own security section heading contains. The context tier then fed the heading to the rule.

Fix. Narrow the rule to the full compound.

Considered and refused. Removing the context tier for that label. The context tier is what makes the hostile tier tractable at all, and removing it to fix one over match would have cost far more than it saved. This defect and the one above were both found by sweeping the generated corpus and cross checking every declaration finding against the generator's own provenance, which is a check worth keeping.

A rule that cannot fire, kept anyway

Symptom. The specification names the Japanese postal mark among the strings the postal code label should match. It never fires.

Diagnosis. Normalisation treats a lone symbol character as a delimiter, so the mark never reaches a token stream. The Japanese postal field is reached through its word form instead, which is why nothing was ever visibly broken.

Fix. The rule stays in the table, with a comment saying it cannot fire and why, and a test asserting both halves of that: that the mark does not survive normalisation, and that the field is still reached by the word form.

Considered and refused. Deleting the rule, or making it reachable. Deleting it would lose the record that the specification asks for something this normalisation cannot deliver. Making it reachable means changing what counts as a token character, which changes every golden snapshot and every committed fixture expectation, and that is a deliberate change with a cost rather than a quiet fix. It is written up as future work rather than done under cover of a different phase.

The tie is the feature

Symptom. Not a defect. It is recorded here because the first design was heading for one and the correction changed the shape of the whole rule table.

Diagnosis. German writes one phrase for both a whole street address and the first of several address lines. French does the same. A field labelled only *Password* is a login field on one page and a registration field on the next. Writing one pattern per label and hoping they do not overlap produces, in each of those cases, a confident answer to a question the evidence does not settle.

Fix. Make the overlaps deliberate. Both labels carry the same phrase at the same weight, the tier ties by construction, and the engine falls through to a tier that might separate them. A tie surviving every tier is `UNKNOWN`. The property that a correct page produces no finding above the lowest severity fell out of this rather than being tuned into existence, and it passed the first time it was run across the entire correct markup slice.

Considered and refused. Breaking each tie by picking a favourite. It would have looked like better coverage on every metric except the one that matters, and it would have produced a confident instruction to write the wrong token into somebody's production markup.

Chapter 4

Generator determinism

Same seed, same bytes. The gate is the generator run twice with a recursive diff of the two outputs, and it is a gate rather than a test because a determinism defect that reaches the corpus invalidates every measurement taken afterwards.

Two of the entries below are defects that had not yet caused a wrong byte and would have, and those are the dangerous kind.

A shared value provider was a latent determinism defect

Symptom. None yet, which is the point.

Diagnosis. The first implementation cached one value provider per locale and reseeded it per form. That is safe only while exactly one provider is alive at a time. Nothing in the call path broke that invariant, which is what made it dangerous: it would have broken the same seed same bytes rule silently, the first time a caller built two forms concurrently, and the failure would have appeared as a corpus that did not reproduce rather than as an exception anybody could trace.

Fix. Each provider owns its own instance. Constructing one costs well under a millisecond, so the whole grid pays a fraction of a second for an invariant that cannot be broken from outside.

Considered and refused. Documenting the constraint and relying on callers to respect it. A documented invariant that only one code path currently satisfies is an invariant waiting for a second code path. Found by a test asserting that two providers built with the same seed produce the same value, which they did not.

The mixed tier could produce a form with no clean section

Symptom. Two templates out of the twenty five then in the grid produced uniformly hostile forms carrying a mixed tier label.

Diagnosis. The mixed tier draw forces at least one hostile section and at least one clean one. The clean pass looked for a section that was not already hostile, and when every section had drawn hostile there was no such candidate, so it silently did nothing.

Fix. A section may now be promoted to clean unless it is the only hostile one, and because a template is required to carry at least two sections there is always a legal choice.

Considered and refused. Redrawing until a legal configuration appears. That changes the number of draws taken, which changes the state of the generator, which changes every subsequent form. A repair that consumes a variable amount of randomness is a determinism defect wearing a fix's clothing.

The frequency is the part worth keeping: two templates in twenty five is exactly the rate at which a defect gets shipped. The parametrised test covers all templates rather than a sample, which is how it was caught.

Infinite recursion comparing two elements

Symptom. A stack overflow while resolving a positional selector in the small HTML tree used to check that generated selectors resolve.

Diagnosis. Every element in that tree carries a pointer to its parent, and the element type was a plain data class with a generated equality method. Resolving a positional selector asks a list for the index of an element, which compares elements, which compares parents, which compares children, and so on up and down the tree.

Fix. Elements are identity objects. Equality generation is switched off and the reason is in the class docstring, so nobody switches it back on for convenience.

Considered and refused. Excluding the parent pointer from the comparison. That produces an equality that says two structurally identical elements in different parts of the tree are the same element, which is false and would have produced wrong selector indices instead of a stack overflow. A crash is a better failure than a silently wrong answer.

A module shadowed a data directory

Symptom. Loading the locale tables looked one directory level too high and found nothing.

Diagnosis. The specification names the data directory with a plural noun, and the obvious name for the module that loads it was the same plural noun with a source extension, beside it. Importing the package qualified name then resolves to the module rather than the directory, and asking the resource machinery for the module's files walks to the package root.

Fix. The module is renamed, the directory keeps the name the specification gives it, and the reason is in the module docstring so that nobody renames it back on tidiness grounds.

Considered and refused. Renaming the directory. The directory name is specified and is part of the layout other people read; the module name is not.

Two templates of one family drew the same values

Symptom. Two structurally different templates of the same family, at the same locale, tier and variant, produced identical sample values.

Diagnosis. The specification writes the per form seed's input as the tuple of family, locale, tier and variant. With several templates per family that tuple no longer identifies a form.

Fix. The template identifier joins the digest input and the family stays in it, so the input is a superset of the specification's rather than a replacement for it.

Considered and refused. Replacing the family with the template identifier. Keeping both costs nothing and means a future change to how templates are named cannot silently collapse two families onto one seed.

Selector resolution checked without a browser

Symptom. Not a defect. A design decision that prevents a whole class of them.

Diagnosis. An answer key whose selectors do not resolve is worse than no answer key at all: every downstream measurement silently loses the fields it could not find, and the loss looks like a classifier failure rather than a corpus defect. That check has to run in the phase that generates the keys, and that phase has no browser.

Fix. A small resolver that handles exactly the subset of the selector language this generator emits, and raises on anything else rather than quietly returning no match for syntax it does not understand.

Considered and refused. Requiring a browser in the generator's gate. That would couple the corpus phase's gate to the extractor phase's dependency and would stop the reachability check from being able to check anything on a machine without a browser installed.

A dependency classification that was reversed

Symptom. The value provider library was recorded as a development dependency on the reasoning that the generator runs from a checkout and never from an installed package.

Diagnosis. That was wrong, and it only became visible once the command surface was wired up: corpus generation is a documented subcommand of the shipped console entry point. A documented command that raises an import error on an installed package is worse for a user than one extra dependency is for the package.

Fix. The library moves to the runtime dependency set. Regenerating the lock file afterwards produced a byte identical file, because the lock is compiled with the development extra already and the resolver annotates both cases the same way, so there is no lock commit. That is the right outcome rather than a skipped step: regenerating a lock that does not change should not create a commit.

Considered and refused. A lazy import with a graceful refusal. That buys a smaller package at the cost of a code path that only ever runs on a misconfigured install, which is a code path nobody will test.

Chapter 5

Export fidelity and the featuriser fallback

The phase that expected a fight with a converter and got a different problem instead.

The fallback was the design, not a retreat

Symptom. The plan anticipated a fight with the converter for text vectorizers and named a fallback: move the whole text pipeline into hand written code, export only the linear layer, and take that quickly rather than spending a day coaxing a converter.

Diagnosis. The fight never started, because the feature set was never convertible. Three of the five feature blocks are categorical one hots, option shape booleans and structural buckets, and no standard transformer produces those. Putting the whole featurisation in the graph would have meant writing custom converters for three bespoke transformers *and* trusting the vectorizer converters, which is a larger version of exactly the risk the fallback exists to avoid.

Fix. The hand written featuriser, shared by training and inference. It is the only design in which one implementation serves both paths, which the feature specification required from the beginning.

Considered and refused. Writing the custom converters. The second argument against arrived for free and will matter longer: a vectorizer anywhere in the inference path puts a full machine learning framework on the dependency list of a tool that installs with one command. The route taken means an installed package needs an array library and a runtime and nothing else.

A reimplementation that is a claim rather than an identity

Symptom. A hand written character n-gram analyser is slower than a compiled one, and its faithfulness to the reference analyser is an assertion.

Diagnosis. So assert it, in a test, rather than in a comment.

Fix. A test asserts, over inputs including empty strings, words shorter than the n-gram length, and Japanese text, that this project's enumeration is character for character the reference implementation's.

Considered and refused. Spot checking a few strings. The detail an independent reimplementation gets wrong is the short word rule: a word shorter than the n-gram length yields its padded self once rather than once per length. Getting that wrong silently triples the weight of every two letter token, and it would not show up in a spot check of ordinary words.

An opset probe, and a warning that is recorded rather than suppressed

Symptom. The export is specified to use the newest opset the resolved runtime accepts, determined by probe. The probe walks down from the newest the installed format library defines and takes the first that both converts and opens in a session. The converter warns at every attempt that the requested opset is above the newest it has been tested against.

Diagnosis. The warning is not noise. It is precisely the case where a successful export proves nothing: the converter is saying it has no test coverage for the combination it just produced.

Fix. The warning is recorded in the decision record rather than filtered, and the thing that proves something is the parity gate. That is why the export phase has a gate rather than a deliverable.

Considered and refused. Suppressing the warning and trusting the successful conversion. A conversion that succeeds and produces different numbers is the failure mode the whole gate exists for.

The graph cannot read its own weights

Symptom. Naming the n-grams behind a prediction, which is what the evidence requirement in this project's first law demands, could not be done from the inference path.

Diagnosis. The exported graph is one classifier node followed by a normaliser, and the coefficients, the intercepts and the class order live in the node's *attributes* rather than in graph initialisers. The runtime does not hand a caller a node's attributes. Reading them would have required the full model format library at runtime to produce one line of a report.

Fix. A separate evidence table, holding each class's highest weighted features, written by the same training run that produces the model.

Considered and refused. Adding the model format library as a runtime dependency, or dropping the evidence line from findings. The first inflates every install for one line of output; the second violates the law the tool is built around.

A derived file is a drift risk, so the parity suite reads the weights back out of the graph and asserts that the table agrees with them. A stale evidence file now fails the build rather than naming the wrong n-grams in a finding, which would have satisfied the evidence law in form and violated it in substance.

The first training run was thrown away

Symptom. The manifest of the first full training run recorded that the working tree was dirty.

Diagnosis. The reproducibility rules say a result produced from a dirty tree is marked dirty and is not citable. The flag did exactly what it exists for.

Fix. The run was discarded and repeated from a clean tree.

Considered and refused. Citing it with a note. The flag needed one fix of its own to be honest: the output directory is untracked on a first run, so counting it would have marked every first training run dirty by construction and made the flag mean nothing. It now excludes the output directory and only that.

This entry has a sequel. See the open item in Chapter 11 about the training manifest of the currently shipped model, which is marked dirty for a different and less avoidable reason and was not thrown away.

Chapter 6

Calibration and threshold derivation

The phase where most of the work turned out to be arranging for numbers to be checkable rather than producing them.

A threshold definition whose answer is always zero

Symptom. The low threshold was specified as the smallest confidence at which the near miss band still captures at least half of the recall the high threshold gives up. Written down carefully, that has a degenerate answer.

Diagnosis. Captured recall only ever falls as the threshold rises, so every value below a qualifying one also qualifies, the smallest qualifying value is always zero, and a threshold of zero says nothing.

Fix. The non degenerate reading is the *greatest* qualifying value, and it was written into the prediction file before the model existed.

Considered and refused. Picking whichever reading produced a nicer boundary once both were computable. Writing the reading down first settled a real ambiguity at a point where there were not yet two candidate answers to choose between, which is the only time that choice is free.

One threshold document would have moved every rule engine finding

Symptom. The derived n-gram boundary sits far above the rule engine's confident band. A single pair of thresholds shared by both engines would have moved every rule engine finding on every page, churned every golden snapshot, and done it as a side effect of training a model.

Diagnosis. A confidence scale is a property of an engine. A calibrated probability and a rule tier band are different quantities that happen to be written in the same units.

Fix. The threshold document carries a block per engine at a bumped schema version, the loader takes an engine name, and the command line resolves the engine first and then asks for that engine's block. The rule block is byte identical to what the previous phase committed, and the golden diff after the whole phase is the version string and nothing else, which is the evidence that it worked.

Considered and refused. One global pair, or an inheritance rule for an engine with no block. An engine with no block raises. That was designed to bite when the language model arrived, and it did, which is how that engine's threshold decision came to be made deliberately and in advance rather than by inheriting a boundary derived on a different scale.

A precision of one over a denominator nobody could see

Symptom. The derived block reported that the precision target was met.

Diagnosis. A precision of one over fifteen accusations and a precision of one over fifteen hundred are the same number and are not the same claim. A reader of the file could not tell which they were looking at.

Fix. The derived block records the denominators beside the rates, and the model card says the same thing in words.

Considered and refused. Reporting the rate alone and mentioning the denominator in prose somewhere else. The file is what the runtime reads and what a reader opens first.

Calibration made the summary statistic worse

Symptom. Expected calibration error was lower before calibration than after it (0.0281 against 0.0454).

Diagnosis. The uncalibrated model on this corpus is already close to honest. Refitting a one versus rest sigmoid per class on a development split of one template per family, and then renormalising, moves classes that were already well behaved.

Fix. The calibrator stays, and the model card says the error went the wrong way and by how much. Two reasons: macro-F1 is higher with it (0.7690 before against 0.7866 after), and the argument for calibrating at all is about the number a single finding quotes to a developer rather than about an average gap across a split.

Considered and refused. Dropping the calibrator. Removing a component because its summary statistic got worse, without re-registering a prediction first, is tuning on the thing being measured. The honest move was to keep the design, publish the number that went the wrong way, and say why the design still stands.

The sweep chose the edge of its own grid

Symptom. The chosen inverse regularisation is the largest value the pre-registered grid offered (Table in the main report's classifier chapter).

Diagnosis. Either the grid is too narrow or the model wants less regularisation than expected, and the data cannot distinguish those without a wider grid.

Fix. The grid was not widened. The consequence is recorded in the model card instead: this model may be less regularised than a wider grid would have chosen.

Considered and refused. Widening the grid and re-running. Widening a pre-registered grid because the answer landed at its edge is how a sweep becomes a search for a number. A later phase that revisits it re-registers first.

An abstention that was right by accident

Symptom. A model emitting the unknown label with a high calibrated probability reached the correct outcome, but through the wrong mechanism: the declaration lookup returned nothing, rather than the confidence gating anything.

Diagnosis. Right answer, wrong reason, and a wrong reason is a defect waiting for the code around it to change.

Fix. An explicit branch. When the winning class is the unknown label the reported confidence is zero, exactly as the rule engine reports it, and the calibrated probability of the runner up stays on its own field where the confusion analysis can still read it.

Considered and refused. Leaving it, on the grounds that the outcome was correct. The outcome was correct for a reason that no test asserted and that any refactor of the declaration lookup would have silently removed.

The model's first wrong accusations

Symptom. After the corpus repair, the n-gram engine accused 334 fields of a missing declaration and was wrong about 9 of them. That is the first time in this project that an engine told a developer to add a token the answer key disagrees with.

Diagnosis. Not a regression in the threshold policy. The target precision behind the high threshold is the pre-registered one and the derivation script was run unmodified. What changed is that the derived threshold now sits on a model whose development precision at that point was itself below one, so a test split precision below one is the design working rather than failing.

Fix. Recorded rather than rounded, in the engineering log, the model card and the main report. A precision target is a target, not a guarantee.

Considered and refused. Raising the target until the wrong accusations disappeared. That is tuning a pre-registered constant against the test split, which is the single worst thing available in this project's rulebook.

A class that finally reached the isotonic switchover

Symptom. Not a defect. A prediction that was correct for one corpus and stopped being correct for the next.

Diagnosis. The earlier phase predicted that no class would ever have enough development positives for isotonic regression, and for its corpus that was true of every class. After the repair, the label covering search boxes and consent checkboxes crossed the switchover.

Fix. The per class method is recorded in the model card, so the change is visible rather than implicit.

Considered and refused. Nothing to refuse here. It is in this chapter because a prediction that holds under one corpus and fails under a larger one is exactly the kind of thing that gets quietly forgotten, and the record of it is what makes the next such prediction more careful.

Chapter 7

Language model schema behaviour and serving

The chapter whose most interesting entry is a failure mode that did not occur.

A seven and a half gigabyte model on disk and unreachable

Symptom. The serving runtime was already installed on the host operating system with the model pulled. From inside the Linux subsystem the model blob was visible on the filesystem and the server was unreachable over the network.

Diagnosis. The host installation binds to loopback only, and loopback inside the subsystem is not the host's loopback.

Fix. Install the runtime inside the subsystem and point its model directory at the host's store. The existing blob is reused with no re-pull.

Considered and refused. Re-pulling the model inside the subsystem, or reconfiguring the host server's bind address. The first costs the download again and doubles the disk; the second changes a setting on the developer's own machine outside the repository, which is the kind of environment drift that makes a result irreproducible for the next person.

A keep alive long enough to be convenient and short enough to give the card back

Symptom. A long keep alive holds three quarters of the accelerator for as long as it lasts. A short one puts a model load into the next call.

Diagnosis. These are the two failure modes and they are not symmetric: an occupied accelerator is a nuisance, and a cold load inside a measured run is a corrupted latency distribution.

Fix. A bounded keep alive of a few minutes, an explicit unload when a run finishes, and every latency figure taken against a warmed session so that the reload costs nothing methodologically as long as the warm up precedes the timing.

Considered and refused. An unbounded keep alive. It would have made the measurement identical and left the accelerator occupied indefinitely, which is a decision that belongs to whoever owns the machine rather than to the benchmark.

The engine order in the benchmark is a methodological choice

Symptom. The benchmark runs the language model first, which is not the order the tables present.

Diagnosis. The two classical engines take a couple of minutes each of browser time with no inference call in them. Running the language model last would have meant several idle minutes before its first call, against a keep alive of a few minutes, so the keep alive could expire inside the measured run and put a cold model load into the per field latency distribution.

Fix. Warm immediately before, run it first, and record the ordering and its reason in the run's own notes so that the table's presentation order cannot be mistaken for the execution order.

Considered and refused. Presenting the tables in execution order. The tables are ordered for a reader; the execution is ordered for the measurement, and conflating the two would have made one of them worse for no gain.

The obvious leak check is wrong in both directions

Symptom. The declared attribute value must be dropped from what the model sees, or it reads the answer off the page on every clean tier form. The obvious check is a substring search for the declared token in the serialised payload.

Diagnosis. That check has two failure modes and both are fatal. A field named `email` that also declares an email autofill token would trip it, and that combination is common enough in a clean corpus that the check would have fired on the first real run and been switched off within the hour. Meanwhile a declaration reaching the payload through some derived value would not trip it at all.

Fix. Check *independence* instead: pruning a descriptor and pruning the same descriptor with its declaration stripped must produce identical payloads. That has neither failure mode, and it is asserted on every request rather than reviewed once.

Considered and refused. The substring check with an allow list of tokens that are also common identifiers. The allow list grows until the check means nothing, which is the same failure the traceability checker was designed to avoid.

The keep list is a floor, not a ceiling

Symptom. The specification's list of what pruning keeps omits several signals that the featuriser feeds to the n-gram model.

Diagnosis. Withholding a signal one engine has from the other tilts the comparison exactly as far as handing over the whole document would, in the opposite direction and with a better conscience.

Fix. The rule applied is that the language model sees what the featuriser sees and nothing more. The three named drops stand, because none of them reaches the featuriser either. The deviation from a literal reading of the list is recorded in the prompt module's docstring with the reason.

Considered and refused. Following the list literally. It would have been defensible by citation and indefensible as a measurement, and the specification's own reason for having the list is fairness rather than the list itself.

A threshold block that had to be unflattering

Symptom. The threshold loader raises for an engine with no block, by design. The language model needed two numbers and the specification forbids its self reported confidence from feeding the threshold policy.

Diagnosis. The only pair satisfying both constraints is zero and zero.

Fix. Zero and zero, registered in the prediction file before the run, with both consequences written down in advance: the engine gets no threshold protection and accuses on every committed prediction, and the low confidence finding becomes unreachable for it.

Considered and refused. Deriving a boundary on the self reported scale against a precision target, the way the n-gram engine's was derived. It would probably have raised the engine's finding level precision substantially. That it would have flattered the engine is exactly why it had to be refused before the measurement rather than after, and why the decision is in a committed file with a timestamp rather than in a paragraph written afterwards.

The failure mode the retry ladder exists for did not occur

Symptom. Across 480 requests, 0 needed a retry and 0 failed after one. The registered prediction was that between none and one in twenty would need a retry.

Diagnosis. Passing the response schema with the request lets the server constrain decoding, which appears to remove the failure mode rather than reduce it.

Fix. Report the zero. The retry ladder is still there and is still tested against a dozen recorded transcripts including deliberately malformed ones, so its behaviour is covered without a network and without a key; on this run it never fired.

Considered and refused. Reporting robustness in prose, or removing the untriggered ladder. A benchmark whose most interesting failure mode did not occur is a result, and the honest report of it is the zero rather than a paragraph about how robust the implementation is. Removing the ladder would leave the next serving route, which may not constrain decoding, with no handling at all.

The prompt fingerprint that fails on purpose

Symptom. Editing the system prompt fails a test that holds a digest of the assembled prompt and schema.

Diagnosis. Working as intended. The prompt is part of what a result means, so a silent prompt edit would invalidate every committed language model result without anything going red.

Fix. The correct response is to bump the prompt version, update the literal, and record the change in the changelog. Updating the literal alone is the one thing the test exists to prevent.

Considered and refused. Comparing the prompt loosely, or leaving it untested. The whole reason a result file records a prompt version is so that two results can be compared, and a version that does not change when the prompt does is worse than no version at all.

Cost is null, and null is not zero

Symptom. The cost column is empty for every engine in every table.

Diagnosis. The two classical engines have no provider and the language model ran locally. Nothing was billed.

Fix. Record null. Zero is a measurement and the electricity was not free; null is the absence of one. The price table document in this repository deliberately contains no prices, because no vendor page was retrieved during this work and a remembered price is a fabricated number.

Considered and refused. Writing zero, or computing a list price estimate from a remembered rate card. The second is worse than the first: it produces a plausible number with a currency symbol on it, which is exactly the shape of claim that survives review.

The evaluation runner pays for the language model twice

Symptom. The run made twice as many requests as there are forms, and about half its wall clock is a second inference pass.

Diagnosis. The evaluation runner classifies every form twice: once directly, which is where the run log's predicted label and per field latency come from, and once through the audit engine, which is where the finding codes come from. For the two deterministic engines the passes agree by construction and cost microseconds. For a language model it doubles the inference.

There is a second consequence and it is the one that matters to a reader of the finding level tables. A row's predicted label and its finding codes come from two different draws of a model that is not guaranteed to be deterministic. Measured on this run, using the deterministic rule engine as a control for the check's own edge cases, the excess disagreement is on the order of two fields in seven hundred.

Fix. Not fixed. Documented in the main report's limitations chapter, in this report, and in the future work list, with the measured magnitude.

Considered and refused. Fixing it in the phase that found it. Fixing it changes what the evaluation runner does for every engine, and doing that after seeing the results is the regeneration this project's reproducibility rules forbid. The fix is to pass the first pass's predictions into the audit rather than letting the audit reclassify, which also turns the two numbers into one fact, and it belongs to a phase that re-registers its predictions and re-runs everything.

Chapter 8

Friction at the package boundary

The evaluation harness is consumed here as an ordinary released dependency: not vendored, not submoduled, not copied file by file. That separation is a deliberate engineering claim, and the friction it exposes is the point rather than an inconvenience. Vendoring the code would have destroyed the evidence by making the boundary unobservable.

Every entry below has an issue filed on that repository with a concrete API proposal, and every workaround that stands here in the meantime is labelled in every result file it produces, because a workaround that is invisible in the output is indistinguishable from a capability.

The anticipated friction was aimed one layer too low

Symptom. The ledger predicted, before the integration began, that categorical outcome support in the permutation machinery would be the problem. It was not needed at all.

Diagnosis. The p value estimator never sees an outcome. It sees an observed effect and a null distribution, both numbers. The categorical part of this project's problem lives entirely in the statistic, and the statistic is the caller's. What actually broke was one layer above, in the data model.

Fix. Record the miss in the ledger rather than quietly replacing the prediction with what turned out to be true.

Considered and refused. Rewriting the anticipation to match the outcome. A ledger that only ever records correct anticipations is a ledger nobody can learn anything from.

The ingestion layer refuses a run log that has no time axis

Symptom. The harness's parser claims a run log of this project's schema and then refuses it, reporting that the records carry no step field and naming the several spellings it would accept.

Diagnosis. There is no step to add. The file has one row per classified field and no time axis at all. Inventing a step from the row index would let the harness's window comparison treat a set of unrelated fields as a stationary process, which is a stronger and false claim.

Fix. Filed with a concrete proposal. In the meantime the bridge reads its own file format, which is a few lines, and nothing is reshaped at the last moment to fit an API that does not quite fit.

Considered and refused. Emitting a synthetic step column. It would have made the parser accept the file and would have made every subsequent analysis silently wrong.

No public entry point accepts a cluster assignment

Symptom. The resampling unit is non negotiable in this project's design, and neither of the harness's comparison modes is clustered. Neither is paired, and both reduce a metric series to a final window mean.

Diagnosis. Three separate assumptions, all of them consequences of the harness having been extracted from a program that observed one training run over time.

Fix. The clustered null is built on this side and handed to the harness's p value estimator, which is the one part of the interface that accepts a caller supplied null. The estimator and the correction stay on the far side of the boundary. The manifest of every analysis records that this happened, in its notes and in a block that names what the harness was and was not used for.

Considered and refused. Degrading silently to unclustered resampling. That would have made every comparison look more significant than it is, and it was the one outcome ruled out before the integration began.

A verdict attached to the wrong comparison, and nothing raised

Symptom. The first cross check run attached the verdict for one metric to a row labelled with a different one. Every value in the row was plausible.

Diagnosis. The harness's classification entry point ends by returning its findings in severity order rather than input order, and the tag on a finding is not a unique key when one metric is compared across several slices, which is exactly this project's family. A positional zip therefore attaches every verdict to the wrong comparison, and because every value is plausible, nothing raises.

Fix. The bridge rejoins on the identity of the result object each finding carries. Caught by reading one line of output that should have said one metric name and did not.

Considered and refused. Sorting both sides by tag before zipping. The tag is not unique, so that produces a different wrong answer. Filed as its own issue with two proposed fixes, because joining on object identity is not a contract worth depending on and the fix belongs on the other side of the boundary.

A relative gate where an absolute one was pre-registered

Symptom. The harness's practical effect threshold is a relative percentage. This project pre-registered an absolute difference in the metric.

Diagnosis. A relative gate turns one pre-registered rule into a different rule in every slice, because the same relative percentage means something different against a large baseline than against a small one.

Fix. The practical gate is applied here, and the harness's classification runs alongside as a visible cross check so that the two verdict columns can be compared. On this data the gap changed no verdict: 0 of 33 comparisons disagreed.

Considered and refused. Adopting the relative gate because it happens to agree on this dataset. A gap that does not bite on one dataset is still a gap, and the dataset it would bite on is any slice with a small baseline, which is most of what a wider benchmark would add.

A truthful mode string the harness cannot label

Symptom. The comparison result's mode field is an unvalidated string, so the bridge writes what actually ran. Two of that type's derived members look the value up in a table holding the two training modes and raise on anything else.

Diagnosis. The type models a closed set of modes in its derived members and an open one in its field.

Fix. The bridge does not call those two members and serialises what it needs itself.

Considered and refused. Writing one of the two known mode strings so that both members work. That would make the derived members work and would be a false statement about which test was run, printed in the result file. This one is deliberately not filed on its own: it is a consequence of the same modelling assumption as the clustering issue, and if a paired clustered mode lands there it will bring its own label with it.

No typing marker, so every value crossing the boundary is unchecked

Symptom. Strict type checking refuses to look inside the package, so every value that crosses the boundary arrives untyped.

Diagnosis. The source is thoroughly annotated. The marker file is the whole fix and it is missing.

Fix. A configuration override in this project's checker settings, filed on the other side, with the cost recorded: the bridge is the one module in the source tree whose external calls are unchecked, which is another reason for it to be the only module that makes them, and a test walks every source file and fails if a second importer appears.

Considered and refused. Writing type stubs here. Stubs for somebody else's package, maintained in a consumer's repository, go stale silently and would have hidden exactly the kind of API change the boundary exists to expose.

A statistics dependency that drags in a logging stack

Symptom. Installing the harness pulls more than a dozen transitive packages, including a training metrics logging stack, a dataframe library, a plotting library and a web application toolkit, for a consumer that uses three functions.

Diagnosis. The harness's ingestion layer reads event files and its reports draw plots, and neither is separable at install time.

Fix. The dependency sits in the development extra rather than in the runtime dependencies. The consequence is stated in the packaging file, in the ledger and in the main report rather than left to be discovered: **the shipped tool cannot compute its own significance tests**.

Considered and refused. Putting it in the runtime dependencies. An install of a command line auditor that dragged a training metrics logging stack onto a developer's machine, so that a command they will never run could compute a permutation p value, would be the wrong trade. The narrower extra that would fix it properly is proposed in the ledger.

The part furthest from its origin transferred without a scratch

Symptom. Not a defect. The most interesting result of the whole integration.

Diagnosis. The step up correction procedure needed nothing at all. It is a pure function over a list of p values, it is checked in that repository against the worked example from the original paper, and it did the right thing here on the first call. The p value estimator fit because it takes the null as an argument. The verdict vocabulary already contained the three categories this project needs plus a fourth for a design that cannot reach its own level, which turned out to be the category every comparison in the first analysis landed in.

Fix. Say so, in the ledger and in the main report.

Considered and refused. Reporting only the friction. The split runs cleanly through the middle of the package: the parts built around the *statistics* generalised, and the parts built around

the *shape of a training run* did not, because that shape was never a statistical assumption in the first place. It was the shape of the one program the package was extracted from, and that is a sharper finding than either half alone.

Chapter 9

The power repair

The largest single defect in the project was not in any line of code. It was in the shape of the corpus, it was found by arithmetic rather than by a failing test, and it invalidated the ability of an entire phase to certify anything.

A design that could not reach its own significance level

Symptom. Every one of the 11 comparisons in the first analysis came back inconclusive, several of them sitting exactly at the design floor, meaning the observed difference was more extreme than every other arrangement the design permits and was still not significance.

Diagnosis. Pure arithmetic, and it needed no test split measurement at all. The resampling unit is the template. Whole templates are assigned to partitions, and the realised grid put one template per family in the test partition, so the test split held 5 templates. A paired sign flip permutation over that many clusters has 32 arrangements, so the smallest attainable two sided p value is 0.0625, and the pre-registered false discovery level is 0.05.

More resamples do not help. At that cluster count the test is already exhaustive. Only more clusters help.

Fix. Widen the corpus. The test partition now holds 10 templates, giving 1024 arrangements and a smallest attainable p of 0.001953, and 0 comparisons in the headline analysis are reported as inconclusive for design reasons.

Considered and refused. Two other options were on the table and both were refused in writing.

Accept the floor and report every comparison as inconclusive. That leaves the headline experiment answering its question with effect sizes and no significance at all, which is a legitimate paper and a poor engineering result.

Pre-register a different alpha. A level one order of magnitude looser would clear the floor. It would also have been chosen *after* discovering that the original level was unreachable, which is indistinguishable from moving the goalposts however carefully it is worded.

Computing the power before the runs rather than after them

Symptom. Not a defect. The decision that made the entry above reportable.

Diagnosis. The arithmetic above was computed and written into the statistical policy file *before* the test split runs, from the split assignment alone.

Fix. Publish it as a finding.

Considered and refused. Discovering it after the runs came back inconclusive and writing it up then. The two documents would have contained the same sentences and would have had

entirely different evidential value. Writing it afterwards would have been indistinguishable from an excuse.

Every recorded number in the repository moved at once

Symptom. Widening the corpus changes the manifest digest, which invalidates the descriptor cache, which changes the training partition, which changes the model, its vocabulary, its calibration and its evidence table, which changes both derived thresholds, which changes every metric in the model card, and separately changes the test partition, which moves both engines' test split numbers in either direction.

Diagnosis. All of it follows mechanically from one deliberate change, and none of it is a regression. But a reader comparing the project against its own history a month later would otherwise see every number change at once with no stated cause.

Fix. The explanation, the changelog entry and the prediction file are one commit and it is the *first* commit of the phase, before the templates, the regeneration and the retraining. Then the model artefacts and the model card in a single commit, as the model card rule requires. Then the new runs and the new analysis.

Considered and refused. Doing the work and writing the explanation at the end. The ordering of the commits is the part that matters, because it is the part a reader can check.

Three labels the model had never seen

Symptom. Three labels sat at zero rows in the training partition, which means the model had been asked about classes it could only ever get wrong.

Diagnosis. Nothing in the growth rule required a training floor. Every clause was about existence somewhere in the system, not about presence in the partition the model actually learns from.

Fix. A new clause: every label a model can predict must appear in the training partition with at least a full template's worth of rows. The constant is arithmetic rather than taste. A template is generated in six locales and four tiers at one variant, and the held out locale is lifted out of training, so one training template contributes exactly that many training rows for a field it carries in every locale. The constant therefore says *at least one whole training template carries this label*, which is the smallest statement about a label that is not an accident of one locale profile. The new templates were designed to carry the starved labels deliberately rather than to acquire them by luck.

Considered and refused. Dropping the three labels from the model's output space. They are specification tokens that real forms use, and a tool that cannot name a field because its own corpus was thin is a tool with a gap in the middle of its taxonomy.

What deliberately did not move

Symptom. Not a defect. The list is here because it is the part that makes the repair honest.

Diagnosis. A phase that moved a threshold and a corpus at the same time would have produced two changes and no attribution.

Fix. The seed stays where it was set. The held out locale stays the same locale. The excluded partition keeps its rule. The alpha, the false discovery rate, the absolute practical effect threshold, the clustering unit and the target precision behind the high threshold are all exactly what the earlier phases pre-registered.

Considered and refused. Taking the opportunity to adjust anything else while the numbers were going to move anyway. It is the most tempting moment in a project to do that, and it is the moment at which a repair becomes a rewrite.

The old result files were kept

Symptom. Two test split runs in the repository were taken on a corpus that no longer exists in that shape, and their numbers are not comparable to anything current.

Diagnosis. They were correctly taken and correctly reported.

Fix. They stay, unedited. Result files are append only history. The honest description of them is the first, underpowered measurement, and they are cited in exactly one place for exactly one purpose, which is to show what the repair did to each engine, with the caveat attached that the two columns are not two measurements of the same thing.

Considered and refused. Deleting them, or regenerating them against the new corpus. The reproducibility rules forbid regenerating to fix a number, and deleting them would remove the only evidence that the project found its own design defect rather than being told about it.

Chapter 10

The checks, and the checks on the checks

Enforcement infrastructure has a failure mode of its own: a check that cannot pass its own rule gets an exemption, and an exemption is where enforcement starts to rot. Three entries below are about checks that had to be built so that they could survive themselves.

The dash checker was its own first violation

Symptom. The first draft of the checker that forbids two specific characters declared those two characters as string literals, which made the file an immediate violation of the rule it enforces.

Diagnosis. A checker that names what it forbids will contain what it forbids.

Fix. Build the characters from their codepoints at import.

Considered and refused. Exempting the checker from its own scan. The same reasoning applies to the commit message checker, whose blocklist would otherwise have been the one file in the repository carrying every string the repository forbids; its patterns are stored encoded and decoded at import. Two tests do the same thing for the same reason. This is a small point and it is a real one.

A traceability check that states its rule positively

Symptom. The obvious design flags every numeric literal in prose and then excuses most of them.

Diagnosis. The exclusion list grows until the check means nothing. That is not a hypothetical: it is the failure mode of essentially every lint rule that starts by flagging everything.

Fix. State the rule positively. Three shapes count as a measurement: a percentage, a probability shaped decimal, and a number sharing a line with a measurement word. Years, dates, version strings, reference numbers, list markers, code blocks and addresses are never flagged at all. A line is cleared either by a result file reference or by an explicit annotation carrying a written reason, so that waving the check through leaves a written trace with a name on it.

Considered and refused. Flag everything and maintain an exclusion list. Chosen against deliberately, and the reasoning is in the checker's own docstring so that the next person to be annoyed by a false negative knows what the trade was.

The traceability check passed on citations of files that did not exist

Symptom. Until the first result files existed the check could only look for the *shape* of a citation, and a check that matches a shape passes on a citation of a file nobody committed.

Diagnosis. That is the same thing as no citation at all, with more characters.

Fix. A resolve mode that walks the whole chain for every reference: the path must exist, a manifest must sit beside it or above it, the manifest must name a commit this repository can produce an object for, and the manifest must not mark the run dirty. The scan and the resolve are separate so that the scanner's own tests can run over strings with no repository behind them.

Considered and refused. Leaving it as a shape match. It had been green since the first phase and would have stayed green forever, because it never asked the version control system for anything.

The first genuinely red gate, and it was correct

Symptom. The traceability job went red on a push, on a check that had passed locally minutes earlier, reporting that a manifest's commit does not resolve in this repository.

Diagnosis. The job checked out at the default depth of one, so the clone held only the tip commit and every citation of a run taken in an earlier commit failed. The message was true of the clone and false of the repository.

Fix. Full history on that job, the same setting the ancestry job already carried for the same reason: both checks ask questions about history, and history is the input.

Considered and refused. Fixing it from the error message without reproducing it. The failure was reproduced locally with a shallow clone before anything was changed, because a failure that is fixed without being reproduced is a failure that is guessed at.

Worth recording for a second reason. It is the first time a gate in this project went red on a real defect rather than on a deliberately broken branch, so the argument for proving that each gate can fail now has a natural example beside its manufactured ones.

The proof of failure exercise found two real problems

Symptom. Each continuous integration job was deliberately broken on a scratch branch, one at a time, to prove it can go red. Two of the runs found something that was not the intended breakage.

Diagnosis. The packaging job's first attempt assumed the installer would place the console script at a fixed path, which is a guess and was wrong on the runner. Separately, a deliberate taxonomy breakage failed both the reachability job and the test job.

Fix. The packaging job now sets its own installer home and binary directory, which both fixes the assumption and gives the built artefact a genuinely clean environment to install into. The double failure was left alone, because it is the correct outcome: the property is asserted in two independent places on purpose, and a defect that only one of them noticed would mean the other was not doing its job.

Considered and refused. Skipping the exercise on the grounds that all six jobs were green. A gate that has only ever been green is indistinguishable from a gate that always returns green, and the run links for each observed failure are recorded in the repository so that the claim is checkable rather than asserted.

Placeholder modules that sank the coverage gate

Symptom. Measured coverage of the library sat well below the gate at a point when the library was a taxonomy, a small command line surface, and a set of empty module placeholders.

Diagnosis. The placeholders were written with a single future import each. That is one executable statement per file, thirty of them, none ever imported.

Fix. Strip the import. The placeholders then have zero statements, which is what the coverage gate should see: the phases that fill them bring their own tests.

Considered and refused. Lowering the gate for the first phase, or excluding the placeholders in the coverage configuration. Both would have needed to be undone later and one of them would have been forgotten.

A reachability check that does more than it was asked to

Symptom. Not a defect. A deliberate strengthening, recorded because it has a sharp edge.

Diagnosis. The rule that the taxonomy has exactly one definition in code is enforceable mechanically by scanning the source and the scripts for label literals written anywhere else.

Fix. It does that. The scan is limited to tokens that cannot be confused with ordinary prose, meaning the hyphenated specification tokens and the uppercase extras.

Considered and refused. Scanning for every token. Several specification tokens are also ordinary English words, and a check that fires on one of them in a comment is a check somebody turns off. A check that cries wolf is worse than no check, because it trains its reader to ignore it.

A first remote that was abandoned

Symptom. The repository was initially pushed to a remote whose name differed in case and separator from the one intended.

Diagnosis. A naming mistake, caught before any run link had been recorded anywhere.

Fix. The remote was repointed before anything referenced it, so no recorded continuous integration link refers to the abandoned repository.

Considered and refused. Deleting the abandoned remote. The credentials in use do not carry delete scope, so it is left in place and recorded here rather than quietly forgotten.

Chapter 11

What is still open

Everything in the previous chapters was fixed, refused, or recorded as a deliberate decision with a cost. These are the ones that are still live, and they are listed here rather than in an issue tracker because a defect record that stops at the fixed ones is a marketing document.

The training manifest of the shipped model is marked dirty

Symptom. The manifest of the training run that produced the currently shipped model records that the working tree carried uncommitted changes.

Diagnosis. The model was fitted in the same working session that authored the new corpus templates. The templates were in the tree and not yet committed at the moment of fitting, which is the natural order to work in and the wrong order to record in.

Fix. None retroactively. Refitting now to obtain a clean manifest would be regenerating an artefact to fix a flag, which is the same move the reproducibility rules forbid for results, and it would produce a different model from the one every committed evaluation result was measured against.

Considered and refused. Ignoring it, or refitting. What is done instead is to state it plainly in three places: the main report's method chapter, its limitations chapter, and here. What follows from it is bounded and worth being precise about. The model artefacts are committed and are content addressed into every evaluation run manifest, so *which* model produced the headline numbers is not in doubt. The corpus digest in the training manifest matches the digest in every evaluation manifest, so the model was fitted on the same corpus it was evaluated against. What is genuinely weaker is the development split evidence, which rests on a run whose tree state was not pinned to a commit.

The procedural fix is in the future work list and it is one sentence: commit the corpus change first, then train from the clean tree.

The evaluation runner still classifies every form twice

Symptom. Described in full in the language model chapter. The excess disagreement between a row's predicted label and its finding codes is on the order of two fields in seven hundred for a non deterministic engine.

Diagnosis. The runner classifies once directly and once through the audit engine, and for a non deterministic engine those are separate draws.

Fix. Not applied. The fix is to pass the first pass's predictions into the audit rather than letting the audit reclassify.

Considered and refused. Fixing it in the phase that measured it. Changing what the runner does for every engine after seeing the results is a regeneration, and the honest place for the fix is a phase that re-registers and re-runs.

A rule in the table that cannot fire

Symptom. The Japanese postal mark rule, described in the normalisation chapter.

Diagnosis. Normalisation treats a lone symbol character as a delimiter.

Fix. Not applied. Making it reachable means changing what counts as a token character, which churns every golden snapshot and every committed fixture expectation.

Considered and refused. A quiet fix inside an unrelated phase. It is a deliberate change with a cost and it belongs to a phase that pays the cost on purpose.

The published package carries no model

Symptom. An install from the published artefact runs the rule baseline, because no model bundle travels with it.

Diagnosis. Model binaries in a package index artefact are a size and licensing decision that has not been made, and the tool is designed to work without one.

Fix. Documented behaviour rather than a silent fallback: the automatic engine prefers the model when one loads and falls back to the rule baseline with one printed line when none does.

Considered and refused. Failing when no model is present, or shipping a model without deciding the question. The current state has a real cost and it is stated: the default install is not the default engine that the measurements recommend, and closing that gap is on the future work list.

Five cross repository issues, all open

Symptom. Every issue filed against the evaluation harness is still open. One of them, publication of that package to an index, is a hard blocker on tagging a stable release here, because this project's own rule forbids a stable tag while any dependency is a source reference rather than a released version.

Diagnosis. The rule is a gate criterion rather than a preference, and it was written before it became inconvenient.

Fix. Not applied. The ledger records each issue with its status and the workaround, if any, that stands here in the meantime.

Considered and refused. Vendoring the three functions this project uses and removing the dependency. That is the cheap move and it would destroy the evidence the boundary exists to produce, which is the whole reason the dependency is external in the first place. The rule holds even when holding it is the expensive option, which is the only circumstance in which a rule tells you anything.

Nothing here has been measured on a real page

Symptom. Every accuracy number in this project is a measurement on a corpus this repository generated.

Diagnosis. By design, because content from real pages never enters this repository. The cost is that the strongest objection to every accuracy number is that a generated corpus has a generator's habits, and the rule table's vocabulary and the corpus's label text were written by the same person.

Fix. Not applied, and it is the single most valuable thing the project could do next.

Considered and refused. Relaxing the synthetic data rule. The right shape is a small hand labelled set of real pages with a published labelling protocol and a published disagreement rate, which needs no scraping and no stored page content, and it should arrive with its prediction registered first. The honest registration is that the rule table's advantage will shrink.